SIMATS ENGINEERING
ASSESSMENT TOOL II — VISUALIZATION EVALUATION EXERCISE

Course: Data Handling and Visualization	**Code:** DSA0603	**CO:** CO4
Duration: 60 min	**Total Marks:** 50	Reg No: 192324285 Name: Surya JK

COURSE OUTCOME COVERED
CO2 Apply appropriate visualization techniques to analyze time series data, detect trends, seasonality, and cyclical patterns. (BL3)
Activity Tool : Visualization Evaluation Exercise
Weightage: 20%

Code of Conduct:
I Surya JK / 192324285 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Evaluation of Visualization Quality 25%	Critical Analysis 25%	Application of Visualization Principles 20%	Organization 15%	Accuracy 10%	Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

VISUALIZATION EVALUATION EXERCISE QUESTIONS WITH ANSWERS

Basis: The questions and terminology below follow the supplied Assessment Tool II. The source lists the supplied datasets but does not attach their CSV contents here, so exact dataset-dependent numerical answers are not invented where the source does not provide enough information.

Question 1: Individual Time Series & Moving-Average Smoothing

Question: Using Nile.csv, convert the value column into an R ts object (start = 1871, frequency = 1) and plot the raw annual flow as a line chart. Overlay a 5-year moving average. Visually identify any point where the level or variability appears to shift and describe the observation in 2–3 sentences.

Answer: The Nile series shows a notable level shift around 1898: the mean annual flow is visibly higher before this point and lower afterward. A moving average smooths year-to-year fluctuations and makes this change in level easier to see. Thus, the series contains a change in its underlying level rather than only random short-term variation.

R Code:

```
nile <- read.csv("Nile.csv")
nile_ts <- ts(nile$value, start = 1871, frequency = 1)

plot(nile_ts, type = "l",
     main = "Annual Nile River Flow",
     xlab = "Year", ylab = "Flow")

ma5 <- stats::filter(nile_ts, rep(1/5, 5), sides = 2)
lines(ma5, lwd = 2)
```

Question 2: Multiple Time Series Comparison & Dose–Response Curve

Question: Part (a): Plot DAX, SMI, CAC and FTSE from EuStockMarkets on the same time axis after re-indexing each series to 100 at its first observation. Comment on which index shows the strongest overall growth. Part (b): Plot root_length against concentration for ryegrass_doseresponse.csv, fit a smooth dose–response curve, describe the shape, and identify approximately the concentration producing a 50% reduction relative to the control.

Answer: Part (a): Re-indexing to 100 makes the percentage changes comparable even though the original indices have different scales. The exact strongest-growing index should be read from the resulting normalized plot/data because the CSV itself is not attached to the supplied PDF. Part (b): The expected dose–response relationship is decreasing: as herbicide concentration increases, ryegrass root length decreases. The exact concentration giving a 50% reduction cannot be stated reliably without the supplied ryegrass_doseresponse.csv values; it should be obtained from the fitted curve or model at 50% of the control root length.

R Code:

```
library(ggplot2)
library(tidyr)

eu <- read.csv("EuStockMarkets.csv")
long <- eu |>
  pivot_longer(cols = c(DAX, SMI, CAC, FTSE),
               names_to = "Index", values_to = "Value") |>
  group_by(Index) |>
  mutate(Index100 = 100 * Value / first(Value))

ggplot(long, aes(x = seq_along(Index100),
                 y = Index100, color = Index, group = Index)) +
  geom_line() +
  labs(x = "Time", y = "Index (first observation = 100)",
       title = "Re-indexed European Stock Indices")

# Part (b)
dose <- read.csv("ryegrass_doseresponse.csv")
ggplot(dose, aes(x = concentration, y = root_length)) +
  geom_point() +
  geom_smooth(method = "loess", se = TRUE) +
  labs(x = "Herbicide Concentration",
       y = "Root Length",
       title = "Ryegrass Dose–Response Curve")
```

Question 3: Seasonal Decomposition of Time Series (STL)

Question: Using AirPassengers.csv, convert passengers to a monthly ts object (start = c(1949,1), frequency = 12), apply STL with s.window = 'periodic', and discuss whether additive or multiplicative decomposition is more appropriate.

Answer: Multiplicative decomposition is more appropriate for AirPassengers because the seasonal fluctuations become larger as the overall passenger level increases. In an additive decomposition, the seasonal amplitude would be expected to remain approximately constant, whereas the AirPassengers plot shows increasing seasonal amplitude with the trend. A log transformation followed by additive STL is a common way to represent this multiplicative structure.

R Code:

```
library(stats)

ap <- read.csv("AirPassengers.csv")
ap_ts <- ts(ap$passengers, start = c(1949, 1), frequency = 12)

fit <- stl(log(ap_ts), s.window = "periodic")
plot(fit)
```

Question 4: Detrending and Residual Analysis

Question: Using co2_mauna_loa.csv, fit a linear trend (co2 ~ time), subtract the fitted values to obtain a detrended series, plot the original series with the trend and separately plot the detrended residuals, and explain what remains.

Answer: After removing the linear trend, a strong repeating annual seasonal pattern remains in the residuals. This shows that simple linear detrending is not sufficient to remove the systematic structure in the Mauna Loa CO2 series: the data contain both a long-term upward trend and pronounced seasonality. A seasonal model or decomposition is therefore more appropriate for fully characterizing the series.

R Code:

```
co2 <- read.csv("co2_mauna_loa.csv")
co2_ts <- ts(co2$co2, start = c(1958, 1), frequency = 52)

time_index <- time(co2_ts)
fit <- lm(as.numeric(co2_ts) ~ time_index)
trend <- ts(fitted(fit), start = start(co2_ts),
            frequency = frequency(co2_ts))
resid_ts <- co2_ts - trend

plot(co2_ts, type = "l",
     main = "Mauna Loa CO2 with Linear Trend",
     xlab = "Time", ylab = "CO2")
lines(trend, lwd = 2)

plot(resid_ts, type = "l",
     main = "Detrended CO2 Residuals",
     xlab = "Time", ylab = "Residual")
```

Question 5: Forecasting Trends and Cycle/Seasonality Detection

Question: Part (a): Fit an ETS or auto.arima() model to AirPassengers and produce a 24-month forecast with prediction intervals. Comment on whether the forecast seasonal pattern is consistent with historical seasonality. Part (b): Use sunspots.csv to compute and plot the ACF and/or periodogram, identify the approximate cycle length, and compare it with the commonly cited ~11-year solar cycle.

Answer: Part (a): A suitable seasonal forecasting model should continue the recurring yearly pattern visible in the historical AirPassengers data, with higher passenger counts during the same seasonal periods and lower counts during the corresponding off-season periods. The prediction intervals should widen as the forecast horizon increases. Part (b): The sunspot ACF/periodogram should show a prominent cycle close to 11 years, consistent with the commonly cited approximately 11-year solar cycle. The exact peak can vary slightly because the observed solar cycle is not perfectly constant.

R Code:

```
library(forecast)

ap <- read.csv("AirPassengers.csv")
ap_ts <- ts(ap$passengers, start = c(1949, 1), frequency = 12)

fit <- auto.arima(ap_ts)
fc <- forecast(fit, h = 24)
autoplot(fc) +
  labs(title = "24-Month AirPassengers Forecast",
```

```
       x = "Year", y = "Passengers")

# Part (b)
sun <- read.csv("sunspots.csv")
sun_ts <- ts(sun$value, start = 1700, frequency = 1)

acf(sun_ts, main = "ACF of Annual Sunspots")
spectrum(sun_ts, main = "Sunspot Periodogram")
```